



Dynamické stránky v PHP

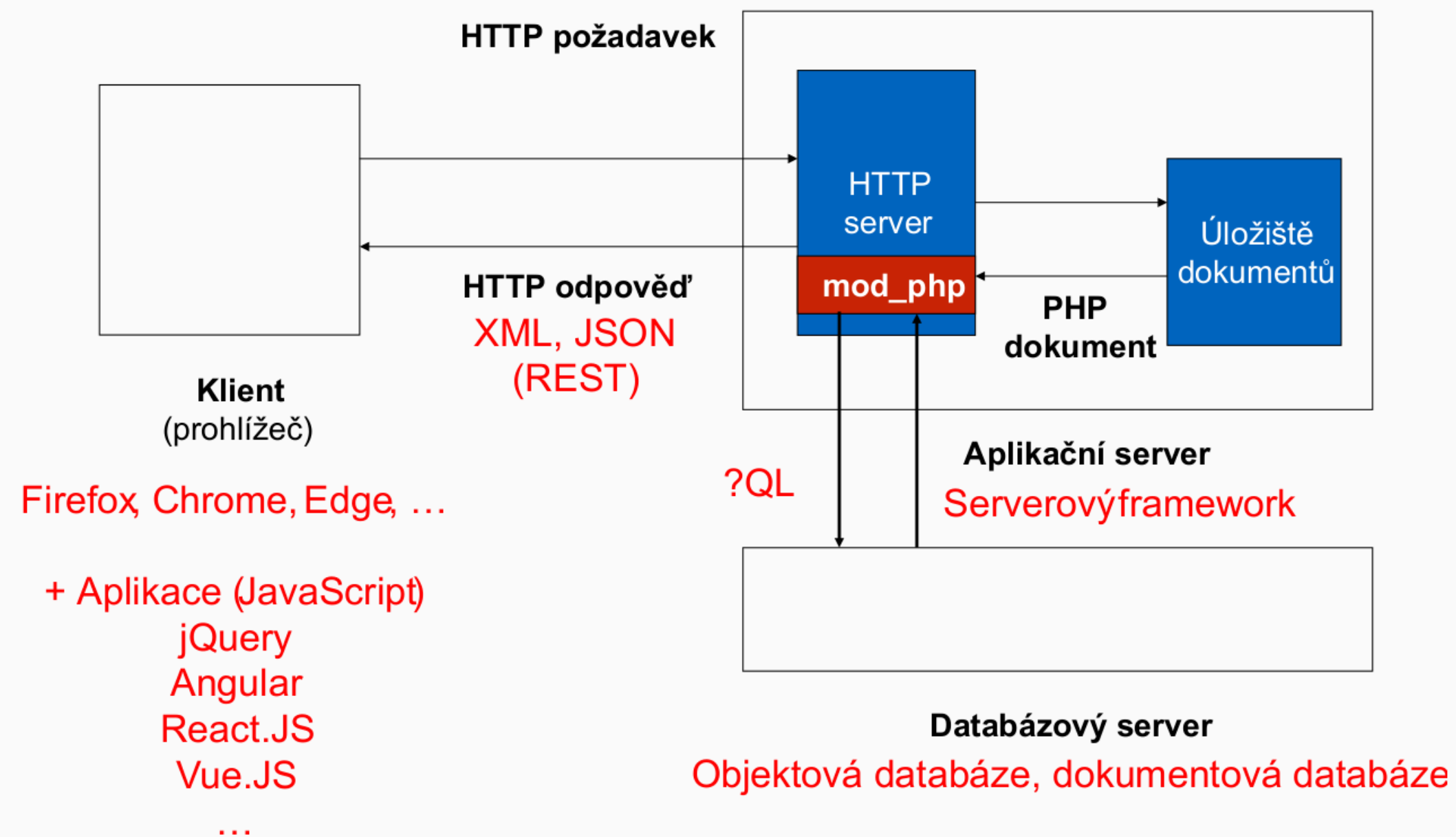
Prezentační a aplikační logika na serveru

Superglobals, HTTP hlavičky, cookies, formuláře, session, autorizace

Ing. Jiří Hynek, Ph.D., hynek@vut.cz

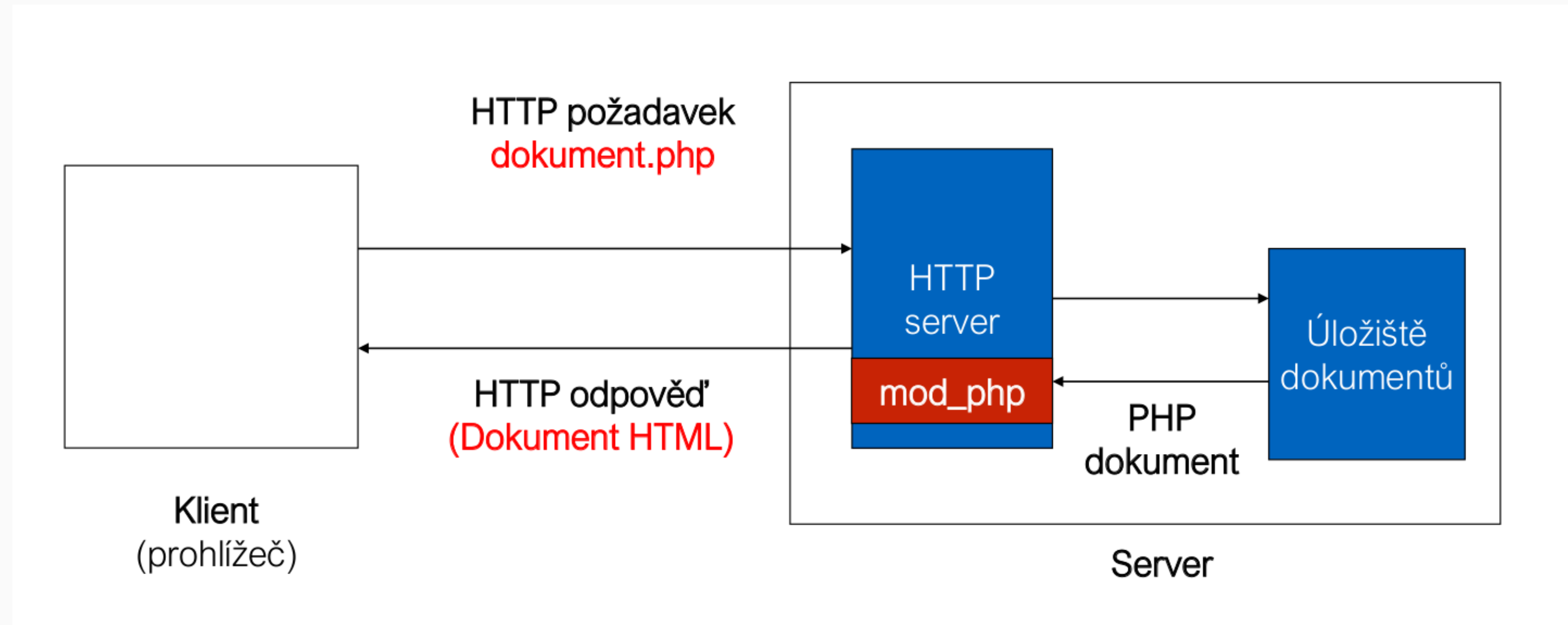
doc. Ing. Radek Burget, Ph.D., burgetr@vut.cz

Architektura



- [Web dev roadmap...](#), [schéma v Excalidraw](#)

Základní architektura v PHP



aplikační vrstva + prezentační vrstva

Obsah

- [Základy jazyka HTML](#)
- [Základy jazyka PHP](#)
- Příklady použití PHP v informačních systémech
 1. [HTTP hlavičky \(headers\), přesměrování](#)
 2. [Cookies](#)
 3. [Zpracování hodnot zaslaných metodou GET a POST](#)
 4. [Session, autorizace](#)

HTML

- HyperText Markup Language
 - značkovací jazyk pro tvorbu hypertextových dokumentů
 - dokument obsahující text v přirozeném jazyce doplněný **značkami** (*markup language*)
 - části obsahu mohou sloužit jako **odkazy na jiné dokumenty** (*hypertext*)
1. Výstup dat – prezentace
 - text (formátování), seznamy, tabulky
 2. Vstup dat – formuláře
 - text, zaškrťovací pole, výběrové prvky
 - doplněné o styly (CSS) a akce (klientský JavaScript)

HTML dokument

- Obvykle označen příponou **.html** nebo **.htm**
- MIME type **text/html**
- Prostý textový soubor (Unicode, kódování **UTF-8**)
- Může obsahovat
 - Text
 - Vložené značky
 - Znakové entity – pro vkládání znaků se speciálním významem:
 - `<`; (<), `>`; (>), `&`; (&), `"`; (")
 - v PHP je při výpisu dat vytváří funkce `htmlspecialchars()`
- Více mezer i konec řádku se zobrazí jako jedna mezera, řádky se zalamují až při vykreslování

Značky a elementy

- HTML element: úsek dokumentu vymezený *značkami*

```
<p>Obsah elementu</p>
```

```
<div class="menu" id="mainmenu">  
Obsah elementu<br> Další obsah elementu.  
</div>
```

```
<div>Nějaký <em>zvýrazněný</em> text.</div>
```

- Element má jméno, atributy a obsah
 - Z definice prázdné ([void](#)) elementy – jen počáteční značka
- Na velikosti písmen u názvů značek a atributů nezáleží (konvence: malá písmena), u hodnot ([id](#), [class](#)) ano

Struktura dokumentu

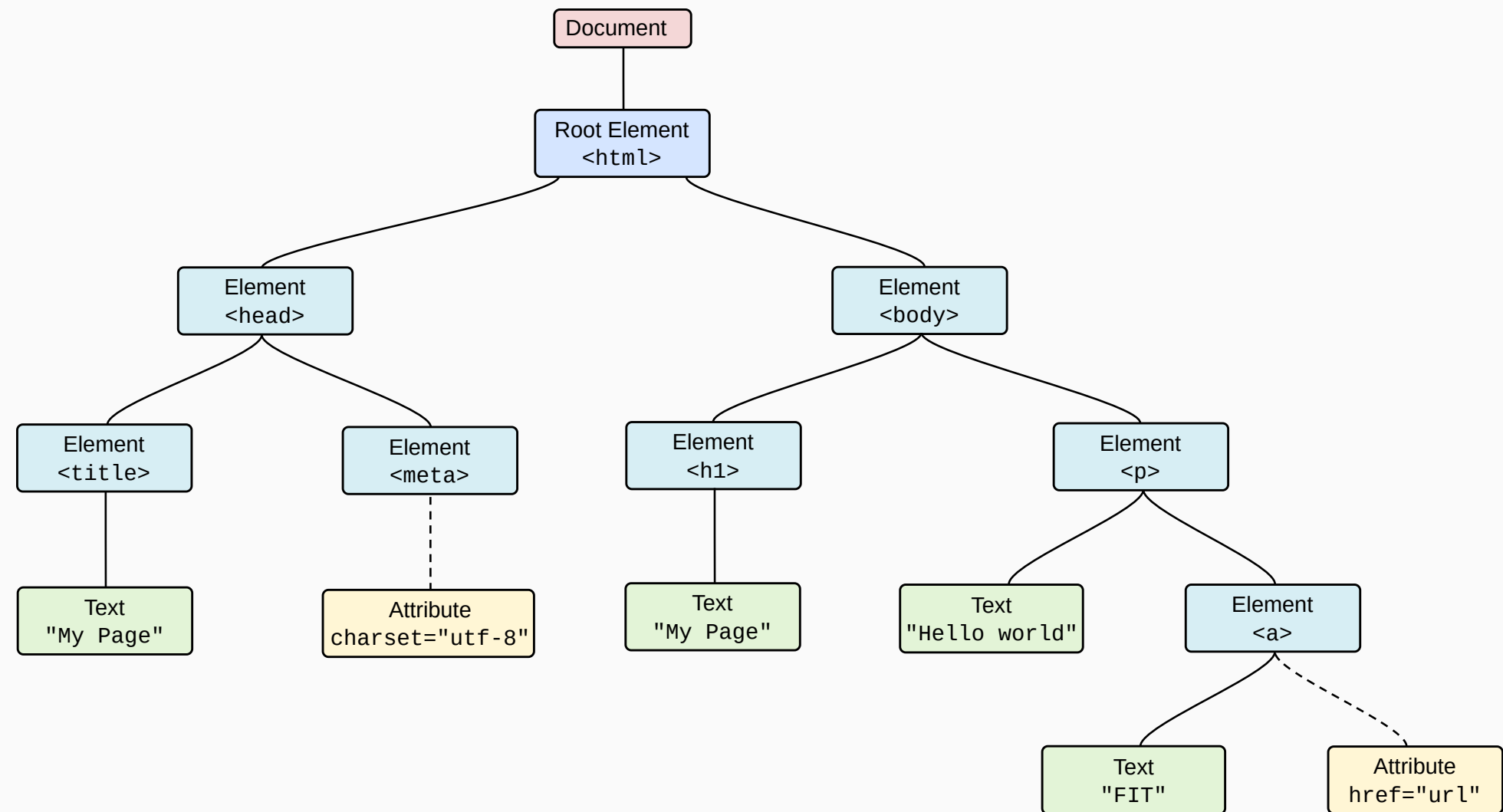
```
<!DOCTYPE html>
<html lang="cs">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Titulek stránky</title>
  </head>
  <body>

    ... tělo dokumentu ...

  </body>
</html>
```

HTML dokument jako strom

- HTML dokument lze popsat jako **strom elementů**
- Kořenový element je tvořen elementem `<html>`
- Synovské uzly jsou tvořeny elementy bezprostředně vnořenými v otcovském elementu
- Atributy (čárkovaně) nejsou potomky, jen patří k elementu



Řádkové a blokové elementy

- **Řádkové** – část textu
 - `<em>`, `<strong>` – zvýraznění
 - `<code>`, `<var>` – kód, proměnná
 - `<sup>`, `<sub>` – horní a dolní index
 - `<a>` – odkaz
 - `<br>` – zalomení řádku
 - `<span>` – obecný řádkový element
- **Blokové** – struktura dokumentu
 - `<header>`, `<footer>`, `<nav>` – záhlaví, zápatí, navigace
 - `<article>`, `<section>`, `<aside>` – článek, kapitola, doplněk
 - `<h1>` – `<h6>` – nadpisy
 - `<p>`, `<ul>`, `<ol>`, `<table>` – odstavce, seznamy, tabulky
 - `<div>` – obecný blokový element

Seznamy a tabulky

```
<ul>
  <li>První položka</li>
  <li>Druhá položka</li>
</ul>
```

- První položka
- Druhá položka

```
<table>
  <caption>Příklad tabulky</caption>
  <tr><th>Jméno</th><th>Příjmení</th></tr>
  <tr><td>Jan</td><td>Novák</td></tr>
  <tr><td>Josef</td><td>Dvořák</td></tr>
</table>
```

Příklad tabulky

Jméno	Příjmení
Jan	Novák
Josef	Dvořák

Globální atributy

- Atributy použitelné pro všechny elementy

Atribut	Význam
<code>id="jmeno"</code>	Jednoznačný identifikátor
<code>class="jmeno"</code>	Použití pro kaskádové styly (CSS)
<code>style="styl"</code>	Styl přímo u elementu (spíše nepoužívat)
<code>data-*="hodnota"</code>	Vlastní data, např. pro JavaScript (<code>data-id="42"</code>)
<code>title="text"</code>	Titulek elementu (zobrazí se v „bublině“)
<code>lang="cs"</code>	Jazyk obsahu
<code>hidden</code>	Skrytý prvek

- Generické elementy `<span>` a `<div>` nemají vlastní význam – určí se pomocí `class`, obvykle s CSS

Formuláře v HTML

- Interaktivní prvek stránky určený pro vstup dat

```
<form action="vysledek.php" method="post">  
    ... HTML kód obsahující definice prvků formuláře ...  
</form>
```

- **action** – dokument zpracovávající vložená data
- **method** – metoda předání dat: GET nebo POST
- Zpracování dat
 - **Na serveru** – klient odešle vyplněné údaje, server je zpracuje a vrátí dokument
 - **Na straně klienta** – JavaScript (validace, odeslání přes `fetch()`)

Vstupní pole

```
<input type="text" name="polozka1" value="obsah">
<input type="password" name="heslo" value="heslo">
<input type="checkbox" name="souhlas" checked>
<input type="radio" name="volba" value="prvni"> První
<input type="radio" name="volba" value="druha"> Druhá
<input type="hidden" name="id" value="42">
<input type="submit" value="Odeslat">
```



☐ První ☐ Druhá

(skrytý prvek se nezobrazí)

Výběr, textová oblast, popisky

- Atribut **for** popisku `<label>` odpovídá atributu **id** vstupního prvku – kliknutím na popisek se prvek vybere

```
<select name="barva">
  <option value="red">Červená</option>
  <option value="blue" selected>Modrá</option>
</select>

<textarea rows="3" cols="20" name="oblast">
Textový obsah
</textarea>

<fieldset>
  <legend>Osobní údaje</legend>
  <label for="jmeno">Jméno</label>
  <input type="text" name="jmeno" id="jmeno">
</fieldset>
```

Modrá ▾

Textový obsah

Osobní údaje

Jméno

HTML5 typy a validace

```
<input type="email" name="email" required>  
<input type="number" name="vek" min="0" max="150">  
<input type="date" name="narozeni">  
<input type="text" name="psc" pattern="[0-9]{5}">
```

email

23

dd.mm.rrrr 

PSČ

- Prohlížeč zkontroluje hodnoty před odesláním formuláře (typ, **required**, **min/max**, **pattern**)
- Validace v prohlížeči **nenahrazuje kontrolu na serveru**
 - požadavek lze odeslat i bez formuláře nebo s upraveným HTML

Formulář a PHP

- Hodnota prvku se odešle pod klíčem daným atributem **name**
 - prvky bez **name** a nezaškrtnuté checkboxy se neodešlou vůbec
- Jméno končící **[]** – PHP z hodnot vytvoří pole

```
<form action="zpracuj.php" method="post">
  <input type="text" name="jmeno">
  <input type="checkbox" name="barvy[]" value="red">
  <input type="checkbox" name="barvy[]" value="blue">
  <input type="submit" value="odeslat">
</form>
```

```
$_POST['jmeno']; // "Jan"
$_POST['barvy']; // ["red", "blue"] -- pouze zaškrtnuté
```

Příklad formuláře

```
<form action="stranka.html" method="GET">
  <label for="name">Jméno</label>
  <input type="text" name="name" id="name"><br>
  <label for="surname">Příjmení</label>
  <input type="text" name="surname" id="surname"><br>
  <label for="stud">Student</label>
  <input type="checkbox" name="stud" id="stud"><br>
  <br>
  <div style="text-align: center">
    <input type="submit" value="Odeslat">
  </div>
</form>
```

Jméno

Příjmení

Student

☐

Odeslat

Odeslání souborů

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="soubor">
  <input type="submit" value="Nahrát">
</form>
```

- Pouze metoda **POST** a kódování **enctype="multipart/form-data"**
 - bez něj se odešle jen název souboru
- V PHP je soubor dostupný v poli **\$_FILES**
 - `$_FILES['soubor']['name'], ['type'], ['size'], ['tmp_name'], ['error']`
 - soubor je uložen v dočasném adresáři – přesun funkcí `move_uploaded_file()`

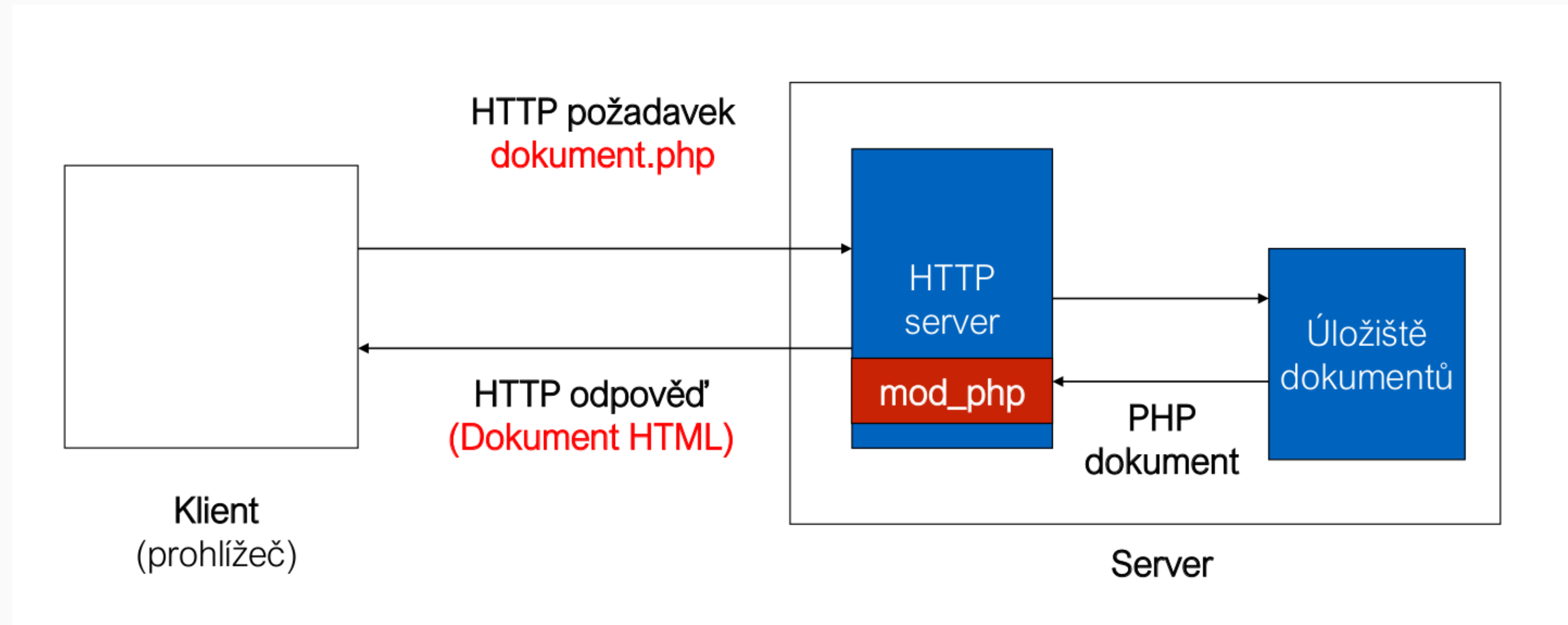
Další zdroje

- předmět ITWe (Web Design):
 - [Introduction to HTML](#)
- online tutoriály:
 - [w3schools.com](#)
 - [MDN Web Docs](#)
 - [jakpsatweb.cz](#)

PHP

- *Personal Home Page*, 1994, později *Hypertext Preprocessor*
- jednoduchý nástroj pro tvorbu dynamických webových stránek
- PHP vychází z následujících hypotéz:
 - problémy řešené na webu jsou často jednoduché, jejich technologické řešení je ale relativně složité
 - WWW stránka je v podstatě pouhý text
- cílem PHP tedy bylo usnadnit vývoj jednoduchých webových aplikací, u kterých se nevyplatí nasadit sofistikované a drahé aplikační prostředí
- k rozšíření PHP přispěla dostupnost na všech klíčových platformách zdarma
- dnes: PHP 8.x, [nejrozšířenější jazyk na straně serveru](#) (WordPress, Laravel, Symfony, ...)

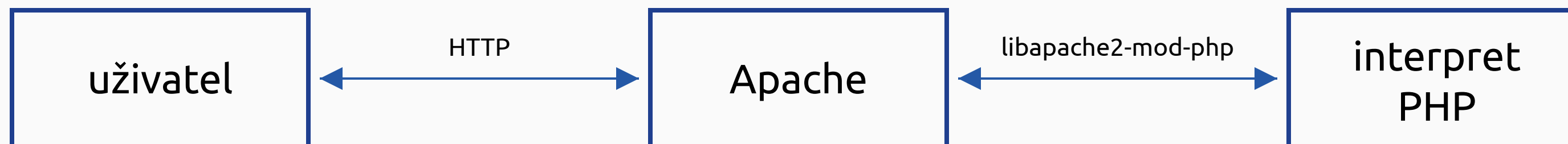
Základní architektura v PHP



aplikační vrstva + prezentační vrstva

PHP: instalace

- instalace aplikačního serveru:
 - Apache (např.: apache2, apache2-utils, ...), Nginx
- instalace PHP a PHP modulu pro aplikační server
 - (např.: php8.4, php8.4-common, ..., **libapache2-mod-php8.4**)
 - nebo samostatná služba **PHP-FPM** (php8.4-fpm) volaná přes FastCGI – obvyklé u Nginx
- konfigurace aplikačního serveru, nastavení složky www
 - (např.: /etc/apache2/sites-available/000-default.conf)



PHP: informace

```
<?php
    phpinfo();
?>
```

- [manuál](#), [ukázka](#)

PHP Version 5.6.40



System	FreeBSD tereza.fit.vutbr.cz 13.0-STABLE FreeBSD 13.0-STABLE #12 stable/13-889517034: Thu Sep 16 09:00:24 CEST 2021 root@tereza.fit.vutbr.cz:/usr/obj/usr/src/amd64.amd64/sys/TEREZA amd64
Build Date	Aug 4 2021 07:39:09
Configure Command	'./configure' '--with-apache=../apache_1.3.41+ssl6' '--with-gd' '--enable-shared' '--with-jpeg-dir' '--with-png-dir=/usr/local' '--with-zlib-dir' '--with-freetype-dir=/usr/local' '--with-iconv=/usr/local' '--with-ldap=/usr/local' '--enable-gd-native-ttf' '--with-openssl=/usr' '--with-mysql=/usr/local/mysql' '--with-mysql-sock=/var/run/mysql/mysql.sock' '--with-xsl=/usr/local' '--enable-exif' '--with-curl=/usr/local' '--with-gettext=/usr/local' '--enable-mbstring' '--with-pdo-mysql' '--with-xmlrpc' '--enable-cli' '--without-pdo-sqlite' '--enable-pcntl' '--with-mysqli' '--enable-zip' '--without-sqlite3' '--with-gd=/usr/local' '--enable-soap'
Server API	Apache
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/lib
Loaded Configuration File	/usr/local/lib/php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20131106
PHP Extension	20131226
Zend Extension	220131226
Zend Extension Build	API220131226,NTS
PHP Extension Build	API20131226,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar, zip
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, sslv2, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk

PHP: vložení do HTML

- Vložení PHP skriptu pomocí značek `<?php` a `?>`
- [tutorial](#)

```
<!DOCTYPE html>
<html>
<body>

  <h1>My first PHP page</h1>

  <?php
    echo "Hello World!";
  ?>

</body>
</html>
```

PHP: generování HTML

```
<ul>
<?php
    echo "<li>první položka</li>\n";
    echo "<li>druhá položka</li>\n";
?>
</ul>
Dnes je <?php echo date("Y-m-d") ?>
<br>
výsledek je <?php echo sin(5);?>

<!-- Konce řádků v kódu != konce řádků zobrazené -->
```

- zkrácený zápis výpisu: `<?= $x ?>` je totéž jako `<?php echo $x; ?>`

PHP: pouze PHP kód

- soubory s příponou .php

```
<?php
function moje_funkce() {
    ...
}

function jina_funkce() {
    ...
}

/* konec PHP záměrně chybí */
```

- uzavírací `?>` na konci souboru se vynechává
 - mezery nebo prázdné řádky za ním by se odeslaly jako výstup a pak už nelze posílat HTTP hlavičky (viz [HTTP hlavičky](#).)

PHP: vkládání skriptů

- [require](#) – příkaz vložení externího skriptu

```
<?php
    require 'somefile.php';
?>
```

- [require once](#) – zkontroluje, zda již soubor nebyl vložen
- [include](#) – totéž, ale chybějící soubor způsobí jen varování (u [require](#) výjimku `Error`, bez zachycení konec skriptu)
- dnes se třídy obvykle nevkládají ručně – [Composer](#) a jeho *autoload* je načítá automaticky (viz [Aplikační frameworky v PHP](#))

PHP: proměnné

- identifikátor proměnné musí začínat znakem \$
- typ proměnné se neudává – určí se podle typu přiřazené hodnoty

```
$jmeno = "Jan";  
$vek = 23;  
$vek = "dostatečný";  
$jmeno_manželky = NULL;
```

- typy parametrů (třída, pole) lze uvádět od PHP 5, od PHP 7 i skalární typy (`int`, `string`...) a návratové typy, od PHP 7.4 typy vlastností tříd (lokální proměnné typ mít nemohou)

```
function secti(int $a, int $b): int {  
    return $a + $b;  
}
```

PHP: řetězce

- V uvozovkách (*double-quoted*)

```
$x = "Nahradí $hodnota a escape sekvence\n";
```

- V apostrofech (*single-quoted*)

```
$x = 'Nenahradí $hodnota a escape sekvence\n';
```

- Spojování řetězců operátorem .

```
$x = 'Jméno: ' . $jmeno . "\n";
```

- Vložení prvku pole do řetězce pomocí { }

```
$x = "Server: {$_SERVER['SERVER_NAME']}";
```

PHP: pole

- představuje mapování klíčů na hodnoty ([dokumentace](#))
 - klíč může být integer nebo string
 - hodnota může být libovolného typu

```
<?php
    $pole["země"] = "Rusko";
    $pole[5] = true;
    echo $pole["země"];
    unset($pole["země"]);
    echo $pole["země"]; // Warning: Undefined array key
?>
```

PHP: pole – funkce

- [count\(\\$pole\)](#) – počet prvků pole
- [in_array\(\\$hodnota, \\$pole\)](#) – obsahuje pole danou hodnotu?
- [array_key_exists\(\\$klic, \\$pole\)](#) – obsahuje pole daný klíč?
- [array_values\(\\$pole\)](#) – hodnoty pole jako nové pole s klíči 0, 1, 2, ...
- [array_keys\(\\$pole\)](#) – klíče pole jako nové pole s klíči 0, 1, 2, ...
- [a spousta dalších...](#)

PHP: pole – konstruktor

- jazyková konstrukce `array()`, nebo zkrácený zápis `[]` (od PHP 5.4)

```
<?php
    $fruits = array(
        "fruits" => array(
            "a" => "orange",
            "b" => "banana",
            "c" => "apple"),
        "numbers" => array(1, 2, 3, 4, 5, 6),
        "holes" => array(
            "first",
            5 => "second",
            "third")
    );
?>
```

```
<?php
    $fruits = [
        "fruits" => [
            "a" => "orange",
            "b" => "banana",
            "c" => "apple"],
        "numbers" => [1, 2, 3, 4, 5, 6],
        "holes" => [
            "first",
            5 => "second",
            "third"]
    ];
?>
```

PHP: pole – průchod

- klíče a příslušné hodnoty

```
foreach ($pole as $klic => $hodnota)  
    echo "Hodnota pro $klic je $hodnota";
```

- pouze hodnoty

```
foreach ($pole as $hodnota)  
    echo "Hodnota je $hodnota";
```

PHP: existence hodnot

- **isset(\$x)** – **true**, pokud proměnná nebo prvek pole existuje a není **NULL**
 - hodnoty z formuláře (**\$_GET**, **\$_POST**) ani cookies nemusí existovat – vždy ověřit

```
if (isset($_POST['login'])) {  
    $login = $_POST['login'];  
} else {  
    $login = '';  
}
```

```
// totéž pomocí ternárního operátoru  
$login = isset($_POST['login']) ? $_POST['login'] : '';
```

```
// totéž pomocí operátoru ?? (od PHP 7)  
$login = $_POST['login'] ?? '';
```

PHP: řídicí struktury

- větvení programu, cykly

```
<?php
    $t = date("H");

    if ($t < "10") {
        echo "Have a good morning!";
    } elseif ($t < "20") {
        echo "Have a good day!";
    } else {
        echo "Have a good night!";
    }
?>
```

- existuje [alternativní syntaxe](#) (`if: ... endif;`), vhodná pro kombinaci s HTML
- dokumentace: [řídicí struktury](#), [if / elseif / else](#), [tutoriál w3schools](#)

PHP: porovnání

- `==` porovná hodnoty po převodu na stejný typ, `===` porovná hodnotu i typ (obdobně `!=` a `!==`)

```
"1" == 1           // true
"1" === 1          // false
null == false      // true
"abc" == 0          // false (od PHP 8), true (PHP 7)
```

- Pozor na funkce, které vrací `false` při neúspěchu, ale jinak mohou vrátit i `0`

```
strpos("/index.php", "/") === 0 // true -- řetězec začíná "/"
strpos("index.php", "/") == 0    // true -- "/" nenalezeno, false == 0
strpos("index.php", "/") === 0   // false
```

PHP: třídy, objekty

- třída

```
class SimpleClass {  
    // property declaration  
    public $var = 'a default value';  
  
    // method declaration  
    public function displayVar() {  
        echo $this->var;  
    }  
}
```

- instance

```
$sc = new SimpleClass();  
$sc->displayVar();
```

PHP: konstruktor

- třída

```
class SimpleClass {  
    public $var = '';  
    public function __construct($value) {  
        $this->var = $value;  
    }  
    public function displayVar() {  
        echo $this->var;  
    }  
}
```

- instance

```
$sc = new SimpleClass('Some value');  
$sc->displayVar();
```

PHP: viditelnost proměnných

Proměnné jsou viditelné v lokálním scope

```
$x = "XXX";

function put_in_x($myparam) {
    $x = $myparam;
}

put_in_x("YYY");
echo $x; // vypíše "XXX"
```

PHP: viditelnost proměnných (2)

- zpřístupnění globálních proměnných pomocí **global**

```
1 $x = "XXX";  
2  
3 function put_in_x($myparam) {  
4     global $x;  
5     $x = $myparam;  
6 }  
7  
8 put_in_x("YYY");  
9 echo $x; // vypíše "YYY"
```

- **global** je spíše nedoporučované – funkce by měla data dostat jako parametry a výsledek vrátit (**return**)

PHP: superglobals

- globální proměnné (asociativní pole) automaticky dostupné ve všech kontextech bez nutnosti deklarace:
 - `$GLOBALS` – globální proměnné
 - `$_SERVER` – informace o hlavičkách, cestách a umístění skriptů
 - `$_GET` – parametry z URL (query string)
 - `$_POST` – data formuláře odeslaného metodou POST
 - `$_COOKIE` – cookies
 - `$_FILES` – soubory poslané pomocí metody POST
 - `$_ENV` – proměnné prostředí, kde běží interpret PHP
 - `$_REQUEST` – obsah `$_GET` a `$_POST` (dle [request order](#) i `$_COOKIE`)
 - `$_SESSION` – proměnné sezení

PHP: ladění

- [var_dump\(\\$x\)](#) – vypíše typ i hodnotu (i pole a objekty)
- [print_r\(\\$x\)](#) – čitelný výpis pole nebo objektu

```
$pole = ["jmeno" => "Jan", "vek" => 23];  
var_dump($pole);    // array(2) { ["jmeno"]=> string(3) "Jan" ["vek"]=> int(23) }  
print_r($pole);     // Array ( [jmeno] => Jan [vek] => 23 )  
echo '<pre>' . htmlspecialchars(print_r($_POST, true)) . '</pre>';
```

- Zobrazení chyb ve stránce – [display_errors](#) v `php.ini`, nebo ve skriptu:

```
ini_set('display_errors', '1');  
error_reporting(E_ALL);
```

- Pouze při vývoji – na produkčním serveru chyby zapisovat do logu ([log_errors](#))

Příklad #1: HTTP hlavičky

HTTP hlavičky

- většinu hlaviček generuje HTTP server automaticky
- funkce [header\(\)](#)
 - `header( 'Content-type: image/png' );`
- hlavičky je možno přidávat jen dokud nebyl zahájen přenos obsahu
 - přenos se zahájí prvním výstupem (např. **echo**)
 - nebo i část dokumentu mimo `<?php ?>` (!)
- funkce [headers sent\(\)](#)
 - vrací true, pokud už byly hlavičky poslány (již nelze definovat další)

Nejpoužívanější HTTP hlavičky

- **Content-Type** – MIME typ obsahu (nejčastěji text/html), např.
 - `Content-Type: text/html; charset=utf-8`
 - `Content-Type: image/jpeg`
 - `Content-Type: application/json` (data pro JavaScript, REST API)
- **Location** – přesměrování na jinou stránku, např.
 - `Location: https://www.jinde.cz/nove/`
- další: **Set-Cookie** (viz [Cookies](#)), **Cache-Control** (ukládání do cache), **Content-Disposition** (stažení souboru)

Příklad přesměrování

```
1 function redirect_to($dest)
2 {
3     if (headers_sent()) {
4         ?>
5         větev klienta
6         <br>
7         <script type="text/javascript">
8             location.href = "<?php echo $dest;?>";
9         </script>
10        nedošlo k přesměrování v javascriptu
11        <br>
12        <a href="<?=$dest;?>">pokračovat</a>
13        <?php
14    } // if headers
```

Pokud byly hlavičky odeslány,
nelze přesměrovat v PHP, ale až
na klientovi
(využití klientského JavaScriptu,
případně HTML značky *meta
refresh*)

Pozor: pokud `$dest` pochází od uživatele, hrozí *open redirect* a XSS – hodnotu ověřit a escapovat podle kontextu (HTML: `htmlspecialchars()`, JavaScript: `json_encode()`)

Příklad přesměrování

```
1  else {
2      $script = $_SERVER["SCRIPT_NAME"];
3      if (strpos($dest, '/') === 0) { // absolute path
4          $path = $dest;
5      } else {
6          $path = substr($script, 0,
7                          strrpos($script, "/")."$dest";
8      }
9      $https = $_SERVER["HTTPS"] ?? "";
10     $scheme = ($https && $https !== "off") ? "https" : "http";
11     header("Location: $scheme://{$_SERVER['HTTP_HOST']}$path");
12     exit();
13 } // else
14 } // of redirect_to
```

Jinak můžeme přesměrovat pomocí funkce `header()`.
Složíme jméno pro přesměrování s využitím **`$_SERVER`**.

Přesměrování dnes

- Hlavička **Location** může obsahovat i relativní adresu ([RFC 9110](#)) – celou URL z **\$_SERVER** není nutné skládat

```
header('Location: vysledek.php'); // relativně k URL požadavku
header('Location: /admin.php');  // od kořene serveru
exit;                             // po přesměrování ukončit skript
```

- Výchozí stavový kód je **302 Found**
- Po zpracování formuláře (POST) je vhodnější **303 See Other**
 - prohlížeč načte cílovou stránku metodou GET a obnovení stránky (F5) neodešle formulář znovu – vzor *Post/Redirect/Get*

```
header('Location: vysledek.php', true, 303);
```

Stavové kódy a výstupní buffer

- **http_response_code()** – nastaví stavový kód odpovědi (200 OK, 403 Forbidden, 404 Not Found, ...)

```
if (!$produkt) {  
    http_response_code(404);  
    echo "Produkt nenaizen";  
    exit;  
}
```

- **Výstupní buffer** – výstup se ukládá do paměti a odešle se až později, hlavičky lze proto nastavit i po **echo**
 - **ob_start()** ve skriptu, nebo **output_buffering** v **php.ini** (často zapnuto, např. 4096 B)
 - proto **header()** po **echo** někdy „funguje“ – nespoléhat na to

Příklad #2: Cookies

Cookies

- malý text uložený v prohlížeči – server ho pošle hlavičkou **Set-Cookie**, prohlížeč ho vrací v hlavičce **Cookie** (jen na daný web a cestu)
- zpřístupněny ke čtení pomocí superglobálu **\$ COOKIE**
 - nová hodnota v něm bude až při dalším požadavku
- zapisují se pomocí **setcookie()** – volat před výstupem (je to **HTTP hlavička**)

```
setcookie("jazyk", // jméno
    "cs",          // hodnota
    time()+3600,    // expirace za hodinu (Unix timestamp)
    "/" );         // cesta
```

- bez expirace obvykle do zavření prohlížeče, smazání = expirace v minulosti, **další parametry**.

Příklad práce s cookies

```
1 <?php
2 $pocet = (int) ($_COOKIE['pocet'] ?? 0); // od klienta → číslo
3 $pocet++;
4 setcookie('pocet', $pocet);
5 if($pocet == 1): // poprvé ?>
6     <h1> Víťame Vás poprvé na našem serveru.</h1>
7 <?php else: // není to poprvé ?>
8     <h1> Jde o Váš <?= $pocet ?> přístup
9     na tuto stránku.</h1>
10 <?php endif; ?>
```

příklad...

Bezpečné cookies

- Od PHP 7.3 lze parametry předat polem

```
setcookie('jazyk', 'cs', [  
    'expires' => time() + 3600,  
    'path'     => '/',  
    'secure'   => true,      // jen přes HTTPS  
    'httponly' => true,      // nedostupné z JavaScriptu (omezí krádež přes XSS)  
    'samesite' => 'Lax',     // cizí weby: jen GET navigace celé stránky (CSRF)  
]);
```

- Cookie může uživatel přečíst i změnit – nevkládat citlivá data (heslo, role) a hodnotám nevěřit
- Omezená velikost (cca 4 KB) – data o přihlášeném uživateli patří do [session](#)

Příklad #3: Zpracování formulářů

Formulář v HTML

Jméno: Příjmení:

```
<form action="phpZpracujForm1.php" method="get">
  <label for="name">Jméno:</label>
  <input type="text" name="name" id="name">
  <label for="surname">Příjmení:</label>
  <input type="text" name="surname" id="surname">
  <input type="submit" value="OK">
</form>
```

Metoda GET

- vyplněné hodnoty se **zakódují do URL dokumentu**, který je zpracovává
- formulář
 - `<form action="zpracuj.php" method="get">`
- vygeneruje URL:
 - `zpracuj.php?name=Jan&surname=Novak`
- PHP dekoduje URL a zpřístupní hodnoty v poli **\$_GET**
 - `$name = $_GET['name'] ?? '';` – hodnota nemusí existovat
 - při výpisu zpět do stránky vždy escapovat: `echo htmlspecialchars($name);`

Metoda POST

- hodnoty se předají **v těle HTTP požadavku** (ne v URL)
 - (požadavek POST místo GET)
- PHP převezme data od webového serveru a zpřístupní v poli **\$_POST** (JSON a jiná data přes [php://input](#))
- **GET jen pro čtení** (vyhledávání, filtrování), **POST pro operace, které mění data** (přihlášení, uložení, smazání)
 - odkaz (GET) může prohlížeč předem načíst nebo ho projde vyhledávací robot

GET vs. POST

- **GET**

- + transparentní
- + hodnoty lze uložit
- + pro odeslání stačí odkaz
- - omezené množství dat
- - nevhodné pro předání citlivých dat

- **POST**

- + velké množství dat (soubory)
- + hodnoty nejsou v URL (chrání až HTTPS)
- - nelze uložit
- - nelze odeslat pouhým odkazem
- - problém s tlačítkem *Zpět* (řeší přesměrování 303)

Příklad: přihlašovací formulář

Login: Heslo:

```
<form action="<?= htmlspecialchars($_SERVER['SCRIPT_NAME']) ?>"
      method="post">
  <label for="login">Login:</label>
  <input type="text" name="login" id="login">
  <label for="pwd">Heslo:</label>
  <input type="password" name="pwd" id="pwd">
  <input type="submit" value="Odeslat">
</form>
```

Příklad: přihlašovací formulář (autentizace)

```
1 <?php
2 $login = $_POST['login'] ?? '';
3 $pwd = $_POST['pwd'] ?? '';
4 if ($login == 'admin' && $pwd == 'blackhat') {
5     // přesměrování na výsledkovou stránku (viz Přesměrování dnes)
6     header('Location: login_vysledek.php', true, 303);
7     exit;
8 }
9 ?>
```

Heslo v kódu je jen pro ukázkou – v praxi se ověřuje proti hashi uloženému v databázi ([password_verify\(\)](#)).

[příklad...](#)

Příklad #4: Session, autorizace

Sessions v PHP

- data o uživateli jsou uložena **na serveru**, klient má v cookie jen náhodný identifikátor session (viz [Bezpečné cookies](#))
- identifikátor session se předává v cookie (výchozí název **PHPSESSID**) – automaticky
- předávání identifikátoru v URL (konstanta **SID**, automatická úprava odkazů) je zastaralé
 - identifikátor uniká v historii, logách serveru a hlavičce **Referer**
 - od PHP 8.4 jeho zapnutí vyvolá varování *deprecated*

Navázání sessions

- před jakýmkoli výstupem je třeba volat **session_start()** (posílá cookie v hlavičce)
- pokusí se načíst identifikátor **PHPSESSID**
 - z cookie
 - pokud žádný není přítomen, vytvoří nový identifikátor
- inicializuje související proměnné

Předávání dat v session

- pole **\$_SESSION** (superglobál)
- před použitím je nutno volat **session_start()**
- hodnoty zapsané do tohoto pole jsou dostupné během celé session

```
$_SESSION["user"] = "root";  
...  
echo $_SESSION["user"];
```

Příklad: autorizace

- chceme, aby některé části aplikace byly dostupné jen přihlášeným uživatelům (login a heslo)
 1. vytvoříme novou session
 2. požadujeme přihlašovací údaje
 3. pokud jsou v pořádku, uložíme do session identifikátor uživatele
 4. chráněné stránky ověří identifikátor uživatele v session
 5. odhlášení = vymazání identifikátoru ze session
- kroky 2–3 = **autentizace** (kdo uživatel je), krok 4 = **autorizace** (co smí)
- [příklad...](#), [demo \(GitHub\)](#)

Příklad: autorizace – přihlašovací dialog (1)

```
1 <?php
2 session_start();
3 $login = $_POST['login'] ?? '';
4 $pwd = $_POST['pwd'] ?? '';
5 if($login == 'admin' && $pwd == 'SpravneHeslo') {
6     session_regenerate_id(true); // nové ID po přihlášení
7     $_SESSION['user'] = $login;
8     header('Location: admin.php', true, 303);
9     exit;
10 }
11 ?>
```

Příklad: autorizace – přihlašovací dialog (2)

```
<html>
<head></head>
<body>
  <form action="<?= htmlspecialchars($_SERVER['SCRIPT_NAME']) ?>"
        method="post">
    <label for="login">Login:</label>
    <input type="text" name="login" id="login"><br>
    <label for="pwd">Heslo:</label>
    <input type="password" name="pwd" id="pwd"><br>
    <input type="submit" value="Odeslat">
  </form>
</body>
</html>
```

Příklad: autorizace – autorizovaná stránka

```
1 <?php
2     session_start();
3     if(($_SESSION['user'] ?? '') !== 'admin') {
4         header('Location: login.php'); // nepřihlášený → přihlášení
5         exit();
6     }
7 ?>
8 <html>
9     ...
10 </html>
```

- přihlášený uživatel bez oprávnění → `http_response_code(403);`
(Forbidden)

Příklad: autorizace – odhlášení

```
1 <?php
2     session_start();
3     unset($_SESSION['user']);
4 ?>
5 <html>
6     ...
7     Odhlášení proběhlo.
8     ...
9 </html>
```

- úplné zrušení celé session: `$_SESSION = []; session_destroy();` – cookie s ID zůstane, smazat přes `setcookie()`

Pokračování příště...